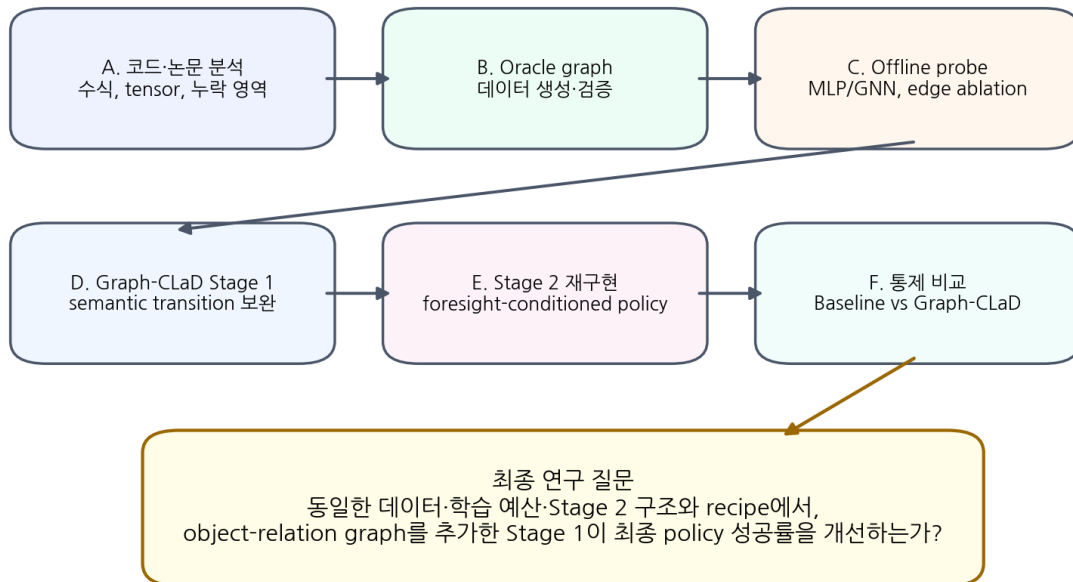


Graph-CLaD 연구 실행 계획서

Oracle Object Graph 기반 Stage 1 검증부터 통제된 Stage 2 Diffusion Policy 비교까지



작성 기준일 2026-08-05

연구 성격 재현이 아닌 통제된 재구현 환경의 아키텍처 비교

문서 사용법과 근거 표시

이 문서는 로봇 학습이나 diffusion policy 에 익숙하지 않은 연구자도 순서대로 따라갈 수 있도록, 개념 설명 → 구현 구조 → 실험 통제 → 일정과 종료 조건의 순서로 구성하였다. 최종 목표는 Stage 2 까지 구현하여 재구현 CLaD 와 Graph-CLaD 를 동일 조건에서 비교하는 것이다. 다만 현재 제공된 코드에는 Stage 2 와 전체 trainer 가 없으므로, 일부 내용은 원 논문의 설명을 바탕으로 새롭게 설계해야 한다.

표시	의미	이 문서에서의 사용
[논문]	CLaD 논문에서 직접 확인	수식, 두 단계 구조, 보고된 주요 hyperparameter
[코드]	제공된 3 개 Python 파일에서 직접 확인	실제 클래스, 함수, tensor 연산, 반환값
[연구 설계]	이번 연구를 위해 새로 제안	oracle graph, GNN, fusion, 통제 비교 protocol
[미확인]	현재 자료로 확인 불가	VLM 전처리, 전체 trainer, Stage 2 network 세부 구조 등

가장 중요한 구분

이 연구의 최종 비교는 논문 표의 94.7%와 직접 경쟁하는 비교가 아니다. 동일한 자체 재구현 환경 안에서 Baseline CLaD 와 Graph-CLaD 를 공정하게 비교하는 연구다. 논문 수치는 참고값으로만 사용한다.

문서 구성

- 1. 연구의 최종 목표와 가능한 주장
- 2. 현재 확보 자산과 누락 영역
- 3. 초보자를 위한 핵심 용어
- 4. CLaD 원 구조를 쉽게 이해하기
- 5. 제공 코드의 실제 데이터 흐름과 tensor shape
- 6. Oracle object graph 를 쓰는 이유
- 7. Oracle graph 명세와 dataset 설계
- 8. Offline graph probe: GNN 이 정말 필요한가
- 9. Graph-CLaD Stage 1 설계
- 10. Stage 2 diffusion policy 구현 목표
- 11. 두 아키텍처를 공정하게 비교하는 protocol
- 12. 전체 실험 matrix 와 metric
- 13. 데이터 split, leakage, seed 와 통계

- 14. 단계별 Gate 와 일정
- 15. repository 와 구현 순서
- 16. 결과 해석과 주장 수준
- 17. 위험 요소와 대체 경로
- 18. 지금 당장 해야 할 작업
- 부록 A. 수식 코드 매핑
- 부록 B. 실험 기록 양식
- 부록 C. 교수님 진행 보고 초안

1. 연구의 최종 목표와 가능한 주장

1.1 최종 연구 질문

최종 연구 질문

동일한 데이터, 동일한 학습 예산, 동일한 Stage 2 policy architecture 와 training recipe 를 사용할 때 object-relation graph 를 Stage 1 에 추가한 Graph-CLaD 가 재구현 CLaD 보다 미래 표현 예측과 최종 LIBERO policy 성공률을 개선하는가?

1.2 연구 가설

1. CLaD 의 compact semantic latent 는 장면의 의미를 담지만 개별 object identity 와 object-object relation 을 명시적으로 보존하지 않을 수 있다.
2. Simulator 에서 얻은 정확한 object identity, pose 와 relation 을 graph 로 표현하면 관계 변화가 중요한 구간의 미래 예측이 좋아질 수 있다.
3. Graph 로 개선된 Stage 1 foresight 가 실제로 유용하다면 동일한 Stage 2 diffusion policy 조건에서 최종 rollout 성공률이나 특정 relation-critical task 성능도 개선될 수 있다.

1.3 가능한 주장과 불가능한 주장

구분	문장 예시	판정
가능	동일한 재구현 조건에서 Graph-CLaD 가 baseline 보다 task 별 평균 성공률이 높았다.	가능
가능	Correct-edge GNN 은 shuffled-edge GNN 보다 relation-change F1 이 높았다.	가능
조건부	Offline 향상이 Stage 2 성공률 향상으로 이어졌다.	Stage 2 까지 완료하고 여러 seed 가 있을 때 가능
불가능	Graph-CLaD 가 공식 논문의 CLaD 94.7%를 능가했다.	공식 전체 코드와 평가 protocol 이 없으므로 불가능
불가능	Oracle graph 성능은 실제 RGB 기반 graph 성능을 보장한다.	Oracle 은 privileged-state upper-bound probe 이므로 불가능

2. 현재 확보 자산과 누락 영역

2.1 확보된 것

자료	확인 가능한 내용
----	-----------

자료	확인 가능한 내용
CLaD 논문 PDF	Stage 1/Stage 2 구조, Eq. (5)-(22), 보고된 hyperparameter 와 ablation
Attentions.py	MultiheadAttention 기반 CrossAttnBlock, SelfAttnBlock
MlpResNet.py	MlpResNet, FiLM layer, FilmMLP
LatentDynamics.py	Stage 1 transition encoding, asymmetric cross-attention, pooling, EMA target, latent/reconstruction loss 반환
기존 연구계획 PDF	Oracle graph 와 MLP-GNN offline 비교를 먼저 수행하는 단계적 계획

2.2 현재 없는 것

- 전체 Stage 1 training loop: optimizer, scheduler, loss 기중합 batch 구성 checkpoint 저장 규칙
- VLM feature 생성 pipeline: 이미지 전처리, frozen encoder, language embedding, view 구성
- LIBERO dataset loader 와 simulator wrapper 의 실제 프로젝트 코드
- Stage 2 diffusion policy network, observation encoder, training loop 와 rollout evaluation 코드
- 저자 pretrained checkpoint 와 공식 configuration 전체

이 누락이 의미하는 것

Stage 2 를 최종 목표로 삼는 것은 가능하지만, 제공 코드의 반칸을 그대로 복원하는 것이 아니라 논문 설명을 만족하는 통제된 재구현을 설계해야 한다. 따라서 모든 새 구현은 baseline 과 graph variant 에 똑같이 적용해야 한다.

3. 초보자를 위한 핵심 용어

용어	쉽게 말하면
Latent	이미지나 상태를 그대로 쓰지 않고, 모델이 압축해서 만든 숫자 벡터. 사람이 직접 해석하기 어려울 수 있다.
Transition	과거 상태와 action 을 보고 현재 또는 미래 상태가 어떻게 바뀌었는지를 표현하는 것
Foresight	미래 관측을 직접 생성하는 대신, 미래 상태를 나타내도록 학습한 latent vector.
Cross-attention	한 정보가 다른 정보를 참고해서 자신을 업데이트하는 연산. CLaD 에서는 proprio transition 이 semantic transition 을 참고한다.
EMA target encoder	학습 중인 online encoder 를 천천히 따라가는 target encoder. 미래 latent target 을 안정적으로 만든다.
FiLM	조건 벡터가 다른 feature 의 scale 과 bias 를 조절하는 방식. Stage 2 에서는 현재 observation 으로 foresight 를 조절한다.
Diffusion policy	정답 action 에 noise 를 넣고, 모델이 그 noise 를 제거하도록 학습하여 action sequence 를 생성하는 policy.

용어	쉽게 말하면
Object graph	Object 를node 로, object 사이관계를edge 로 표현한 자료구조
Oracle graph	Simulator 의 정확한object ID 와pose 로 만든graph. 실제RGB 에서 추정한graph 가아니다
Ablation	구성요소를 제거하거나 섞어서 성능 향상의 원인이 무엇인지 확인하는 비교 실험

4. CLaD 원 구조를 쉽게 이해하기

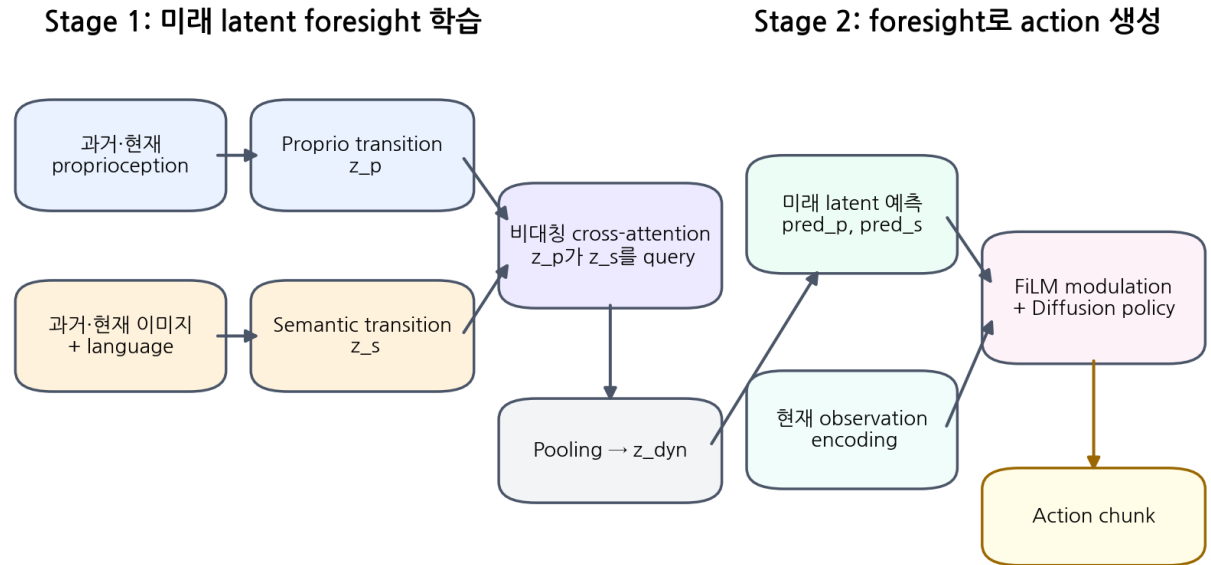


그림 1. 논문 설명을 바탕으로 다시 그린 CLaD 의 두 단계 구조

4.1 Stage 1: 미래 상태를 잘 나타내는 latent 를 학습

[논문] CLaD 는 proprioceptive state 와 semantic state 를 각각 encoding 하고, 과거 상태와 action history 를 참고해 두 종류의 transition representation 을 만든다. 이후 proprioceptive transition 이 semantic transition 을 query 하는 비대칭 cross-attention 으로 공통 dynamics z_{dyn} 을 만든다.

```

z_p = CrossAttn(p_current, [p_past ; action])
z_s = CrossAttn(s_current, [s_past ; action])
z_p_to_s = CrossAttn(z_p, z_s)
z_dyn = LearnablePool(z_p_to_s)

```

z_{dyn} 에서 미래 proprioceptive latent 와 미래 semantic latent 를 예측한다. target 은 EMA encoder 가 실제 미래 관측을 encoding 한 값이다. 또한 predicted latent 가 실제 관측과 연결된 의미를 유지하도록 reconstruction loss 를 함께 사용한다.

4.2 Stage 2: 미래latent 를 이용해action 생성

[논문] Stage 2에서는 Stage 1 이 예측한 미래latent foresight 를 고정된 조건 정보로 사용한다. 현재observation 을encoding 한 뒤FiLM 으로foresight 를 현재 상황에 맞게 조절하고, diffusion policy 가action chunk 를 생성한다.

```
o_p, o_s = Encode(current observation)
g_p = FiLM(pred_p_foresight, o_p)
g_s = FiLM(pred_s_foresight, o_s)
loss_policy = || noise - policy(noisy_action, diffusion_step, g_p, g_s) ||^2
```

중요

Stage 1 이 미래representation 을 만들고, Stage 2 가 그representation 을 사용해 행동한다. 따라서 이번 연구의 핵심은 graph 가 Stage 1 representation 을 개선하는지와, 그 개선이 Stage 2 action 성능까지 이어지는지를 단계적으로 확인하는 것이다

5. 제공 코드의 실제 데이터 흐름과tensor shape

제공된 LatentDynamics.py 기준 Stage 1 실행 흐름

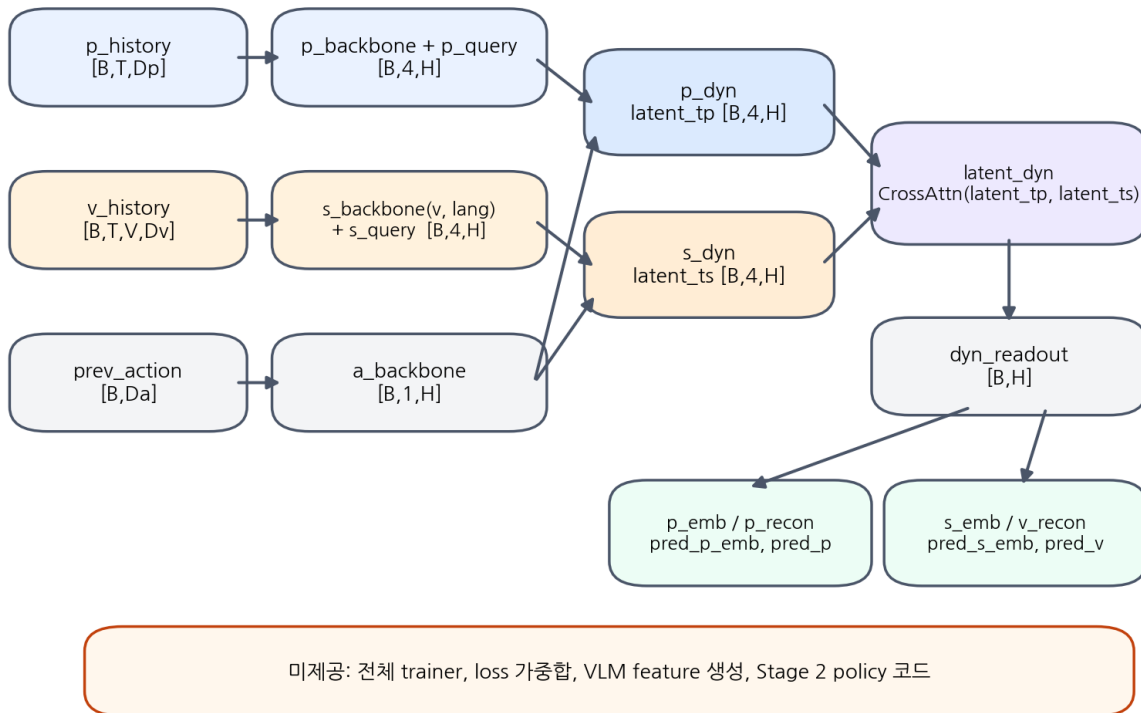


그림 2. 제공된 LatentDynamics.py 의 실행 흐름

5.1 입력과 출력

이름	예상shape	의미	근거상태
v_history	[B,T,V,Dv]	과거 현재 시점의view 별 visual-language feature	[코드]

이름	예상 shape	의미	근거 상태
p_history	[B,T,Dp]	과거 현재 proprioception	[코드]
prev_action	[B,Da]	action history 를 어떤 방식으로든 encode/flatten 한 입력	[코드, 정확한 구성은 미확인]
lang	[B,vl_dim]	language condition feature	[코드]
p_next	[B,Dp]	미래 proprioception target	[코드]
v_next	[B,V,Dv]	미래 visual feature target	[코드]
eval output	pred_p_emb, pred_s_emb	Stage 2 에 넘길 수 있는 미래 latent	[코드]

5.2 논문 수식과 제공 코드의 대응

수식	논문 역할	코드 위치	상태
Eq. 5	$p_t = f_p(p_t)$	p_backbone + p_query	부분 확인
Eq. 6	$s_t = f_s(v_t, l)$	s_backbone(v_flat, lang) + s_query	부분 확인
Eq. 7	proprio transition z_p	p_dyn(p_curr, [p_prev, action])	확인
Eq. 8	semantic transition z_s	s_dyn(s_curr, [s_prev, action])	확인
Eq. 9	z_p 가 z_s 를 query	latent_dyn(latent_tp, latent_ts)	확인
Eq. 10	learnable pooling	dyn_readout(dyn_query, latent_dyn)	확인
Eq. 11-13	미래 latent predictor	p_emb, s_emb	확인
Eq. 14-16	EMA target	update_ema, target backbones	구조 확인, 호출 위치 미확인
Eq. 17	latent prediction loss	loss_p, loss_s	부분 확인
Eq. 18	reconstruction loss	loss_p_recon, loss_v_recon	확인
Eq. 19	total weighted loss	forward 밖에서 합산해야 함	미확인
Eq. 20-22	Stage 2 policy	제공 파일에 없음	미확인, 새 구현 필요

5.3 코드에서 발견되는 주의점

- `s\_backbone`은 input_dim=vl_dim*2 이고 v_history 의 V 개 view 를 flatten 한다. 현재 형태가 성립하려면 $V * D_v = 2 * v_l_dim$ 이어야 한다. 논문 설정과 실제 feature 구성은 실행으로 확인해야 한다.
- `s\_backbone` output_size=vl_dim 인데 s_query 의 마지막 dimension 은 hidden_dim 이다. 덧셈이 되려면 실사용 config 에서 vl_dim 과 hidden_dim 이 같을 가능성이 높지만 이것은 코드 구조에서 도출한 조건이며 config 로 확인되지 않았다.

- Action masking 함수는 concatenated sequence 에서 첫 token 만 보호한다. p_prev 와 s_prev 가 4 token 이면 일부 state token 도 mask 될 수 있다. 의도된 action-only masking 인지 확인이 필요하다.
- Target encoder 는 requires_grad=False 지만 `update\_ema()` 는 forward 안에서 호출되지 않는다. Trainer 에서 optimizer step 이후 호출해야 한다.
- 논문은 normalized latent target 을 사용한다. 제공 코드는 target 만 normalize 하고 prediction 은 normalize 하지 않은 채 MSE 를 계산한다. 원 의도인지 실험적으로 확인해야 한다.
- 네 개 loss 를 어떤 가중치로 합산하는지는 제공 코드에 없다. 논문은 reconstruction coefficient 0.1 을 보고한다.

6. Oracle object graph 를 쓰는 이유

Oracle object graph: 무엇을 입력하고 무엇을 예측하는가

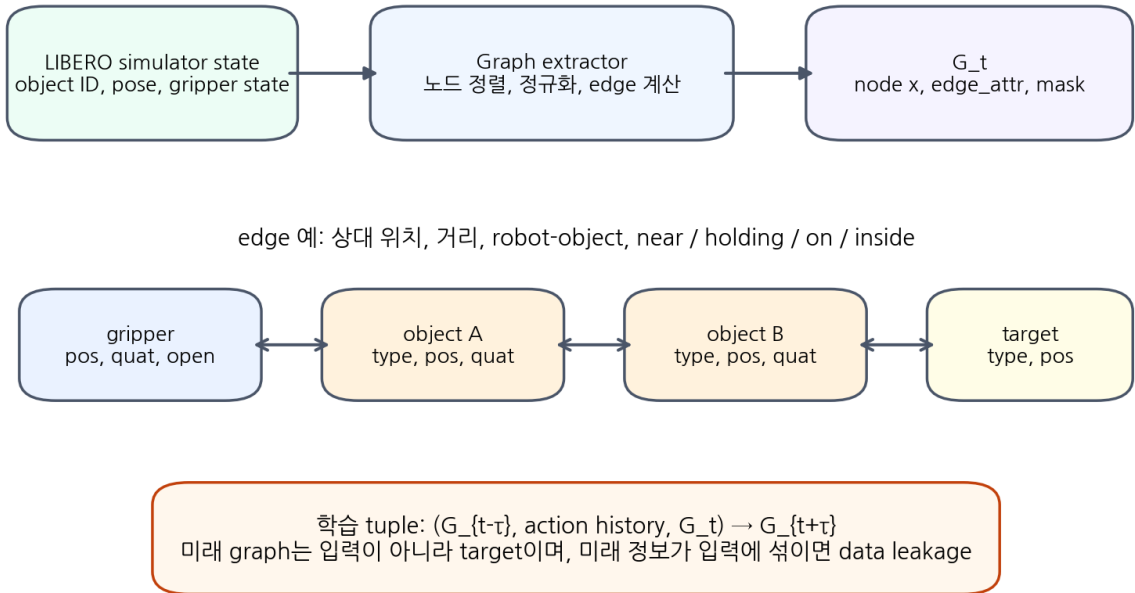


그림 3. Simulator state 에서 oracle graph 를 만드는 개념

6.1 연구 질문을 분리하기 위해

처음부터 RGB object detection, tracking, pose estimation 과 graph learning 을 한꺼번에 구현하면 결과가 나쁠 때 원인을 구분할 수 없다. Oracle graph 는 perception 오류를 잠시 제거하고, 정확한 object identity 와 relation 이 주어졌을 때 graph representation 자체가 도움이 되는지만 확인한다.

분리하려는 질문	Oracle graph 가하는 역할
Representation 질문	정확한 관계 정보가 미래 object transition 이나 CLaD foresight 를 개선하는가?
Perception 질문	RGB 에서 object 와 relation 을 얼마나 정확히 추출할 수 있는가? - 이번 1 차 연구에서는 제외

6.2 후속 연구 투자 여부를 판단하는 upper-bound probe

Oracle graph 에서도 효과 없음

→ graph 구조, target, fusion 위치를 재검토

Oracle graph 에서 효과 있음

→ noisy graph / RGB graph 로 확장할 근거 확보

6.3 Oracle 의 한계

Oracle graph 는 실제 배포 환경에서 직접 얻을 수 없는 privileged information 을 사용할 수 있다. 따라서 oracle 결과는 실제 RGB 기반 성능을 보장하지 않는다. 결과 문장에는 반드시 "simulator-state oracle graph 를 사용한 feasibility study"라고 적는다.

7. Oracle graph 명세와 dataset 설계

7.1 Node 정의

Node 유형	필수 feature	선택 feature	주의
Gripper/end-effector	type ID, xyz, quaternion, gripper open/close	velocity, holding flag	모든 episode 에서 안정적 ID 사용
Task-relevant object	object type, instance ID, xyz, quaternion	velocity, joint/state flag	동일 종류 object 의 instance 를 구분
Target/receptacle	type, xyz, quaternion	open/close, articulated state	task 에 실제로 필요한 target 만 포함

7.2 Edge 정의

Edge feature	형태	설명
relative_position	연속값 3D	receiver 기준 sender 의 dx, dy, dz
distance	연속값 1D	Euclidean distance
pair_type	범주형	robot-object, object-object, object-target 등
near	binary 또는 soft	거리 threshold 기반
holding	binary	gripper-object grasp 상태가 신뢰 가능할 때
on / inside	binary	simulator predicate 나 안정적 geometric rule 이 있을 때

7.3 정규화와 좌표계

- 가능하면 world coordinate 보다 robot base coordinate 또는 task-local coordinate 를 사용한다.
- Position 은 training split 의 평균 표준편차로 normalize 하고 test 정보는 사용하지 않는다.
- Quaternion 은 sign ambiguity 를 주의하고, 필요하면 6D rotation representation 으로 바꾼다.
- Distance threshold 는 task scale 에 맞춰 고정하고, test 결과를 보고 조절하지 않는다.
- Variable object count 는 node mask 또는 batch graph 구조로 처리한다.

7.4 데이터 tuple 과 split

입력 $G_{\{t-\tau\}}$, $action_{\{t-\tau:t\}}$, G_t
 Target: $G_{\{t+\tau\}}$

관장 split: episode 단위 train / validation / test

Leakage 경고

같은 episode 의 인접 frame 이 train 과 test 에 나누면 거의 같은 장면이 양쪽에 들어가 성능이 과대평가된다. 반드시 episode-level split 을 사용한다. 미래 graph, success signal, 종료 step 정보는 입력에 넣지 않는다.

8. Offline graph probe: GNN 이 정말 필요한가

8.1 Stage 2 보다 먼저 해야 하는 이유

Stage 2 rollout 은 구현 비용과 변동성이 크다. Graph representation 이 기본적인 미래 object transition 조차 개선하지 못한다면 복잡한 policy 까지 연결할 근거가 약하다. 따라서 해석 가능한 offline task 에서 GNN 의 필요성을 먼저 확인한다.

8.2 입력과 예측 target

예측 target	Loss/metric	연구 질문
미래 object displacement	MSE, MAE	각 object 의 이동을 얼마나 정확히 예측하는가
미래 object-target relative position	MSE	공간 관계를 보존하는가
Relation change	Macro F1	far→near, near→holding, holding→on 같은 변화를 포착하는가
Instance-specific error	object instance 별 평균	유사 object 사이 identity 를 유지하는가

8.3 최소 비교군

ID	모델	목적
P0	Parameter-matched MLP	graph 구조 없이 같은 입력 정보를 flatten 해 사용
P1	No-edge GNN	node message passing 구조 또는 node encoder 효과 확인
P2	Correct-edge GNN	정확한 relation 의 효과 확인
P3	Shuffled-edge GNN	성능이 edge 의미 때문인지 검증
P4	선택 action 제거	action history 기여 확인

8.4 Stage 1 로 넘어갈 조건

전체 평균이 약간 좋아지는 것만으로는 부족하다. Relation 이 변하는 subset 이나 유사 object 가 여러 개 있는 subset 에서 correct-edge GNN 이 일관되게 우수하고, edge 를 shuffle 하면 성능이 떨어지는 패턴이 가장 설득력 있다.

9. Graph-CLaD Stage 1 설계

권장 Graph-CLaD Stage 1: semantic transition residual fusion

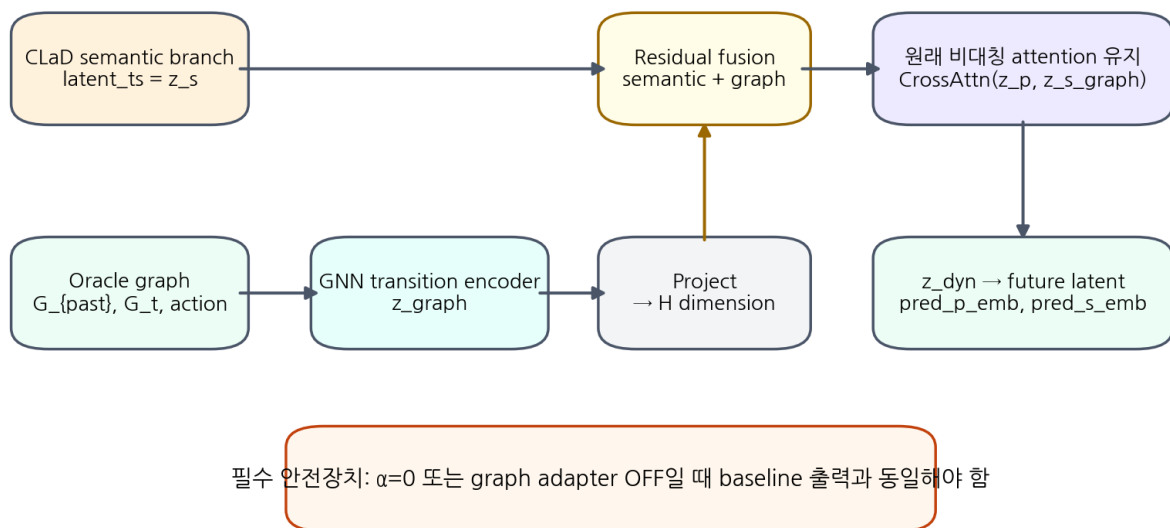


그림 4. 권장 1 차 fusion: semantic transition residual adapter

9.1 Graph 를 어디에 넣을 것인가

후보	장점	위험	우선순위
Semantic transition 전	Visual feature 와 object feature 를 일찍 결합	입력 구조가 크게 바뀌고 baseline 보존이 어려움	낮음

후보	장점	위험	우선순위
Semantic transition z_s 와 fusion	Graph 가 semantic relation 을 보완하고 원 asymmetric attention 유지	token broadcast/정렬 설계 필요	권장
z_{dyn} 뒤에 fusion	구현이 단순	Cross-modal dynamics 학습 자체에는 graph 가 참여하지 못함	진단용

9.2 권장수식

```

z_graph = GraphTransitionEncoder(G_{t-\tau}, action, G_t)
z_graph_h = Project(z_graph)
z_s_tilde = z_s + \alpha * z_graph_h
z_p_to_s = CrossAttn(z_p, z_s_tilde)
z_dyn = Pool(z_p_to_s)

```

9.3 첫 구현 원칙

- Baseline CLaD 코드는 별도 파일로 보존하고 직접 덮어쓰지 않는다.
- Graph adapter 가 꺼졌을 때 $\alpha=0$ 이고 baseline output 과 수치적으로 같아야 한다.
- 처음에는 graph encoder, projection, α /fusion 만 추가한다. 나머지 hidden size 와 token 수는 baseline 과 같다.
- Parameter-matched MLP adapter 를 함께 만들어 추가 parameter 효과를 통제한다.
- Stage 1 의 output interface 는 baseline 과 graph 모델 모두 pred_p_emb , pred_s_emb 형태로 동일하게 유지한다.

9.4 Stage 1 학습 비교

통제 항목	Baseline	Graph-CLaD
데이터 split	동일	동일
VLM 와 proprio input	동일	동일
hidden dimension/token/horizon	동일	동일
optimizer/LR/schedule	동일	동일
training steps/batch size	동일	동일
초기화	공통 모듈은 같은 seed/weight	공통 모듈 동일 + graph branch 추가
Loss	latent + recon	동일 loss + 필요시 graph auxiliary loss 는 별도 ablation

10. Stage 2 diffusion policy 구현 목표

10.1 이번 문서에서 Stage 2 의 위치

Stage 2 는 선택 사항이 아니라 최종 목표다. 다만 Stage 1 과 offline probe 를 통과한 뒤 진행한다. 제공 코드에는 Stage 2 가 없으므로 논문의 Eq. (20)-(22)를 만족하는 별도 policy module 과 trainer 를 구현해야 한다.

10.2 Stage 2 가 받는 입력

입력	출처	예상 역할
pred_p_emb	고정된 Stage 1	미래 proprioceptive foresight
pred_s_emb	고정된 Stage 1	미래 semantic foresight
current proprioception	현재 observation	현재 상태로 foresight 를 조절
current semantic/visual feature	현재 observation	현재 장면으로 foresight 를 조절
noisy action chunk x_k	정답 action 에 noise 추가	diffusion denoising 대상
diffusion step k	무작위 timestep	noise level 정보

10.3 최소 구현 pipeline

```
# Stage 1 은 고정
with no_grad():
    pred_p_emb, pred_s_emb = stage1(history, action_history, language)

# 현재 관측 encoding
o_p = current_proprio_encoder(p_current)
o_s = current_semantic_encoder(v_current, language)

# 현재 상황에 맞게 foresight 조절
g_p = FiLM(pred_p_emb, o_p)
g_s = FiLM(pred_s_emb, o_s)

# Diffusion training
k ~ Uniform(1, K)
eps ~ Normal(0, I)
x_k = add_noise(action_chunk, eps, k)
eps_hat = policy(x_k, k, condition=[g_p, g_s])
loss_policy = MSE(eps_hat, eps)
```

10.4 현재 자료로 미확인인 세부 구현

- Diffusion noise predictor 가 1D U-Net 인지 Transformer 인지 MLP 인지
- Action chunk 의 정확한 dimension 과 horizon 구성
- Current observation encoder 의 정확한 구조와 Stage 1 encoder 공유 여부
- FiLM 을 policy 의 어느 layer 에 몇 번 적용하는지

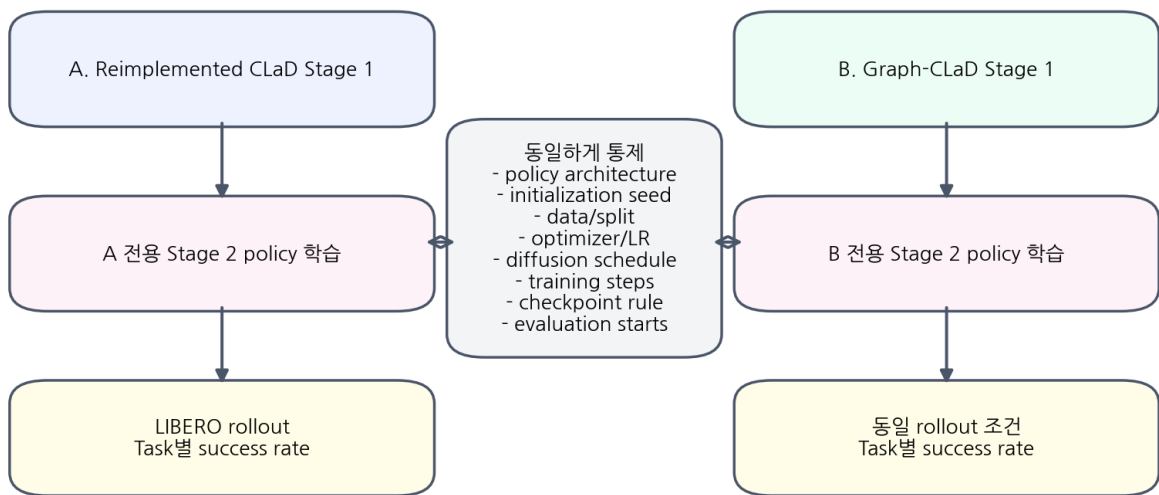
- Noise schedule, sampler, EMA policy 사용 여부와 rollout action execution 방식

설계 원칙

미확인 세부 사항은 하나의 합리적인 구현으로 고정한 뒤, Baseline 과 Graph-CLaD 양쪽에 완전히 동일하게 적용한다. 이 연구의 핵심은 원 논문을 완벽히 복원하는 것이 아니라 representation 차이를 통제된 policy 에서 비교하는 것이다.

11. 두 아키텍처를 공정하게 비교하는 protocol

Stage 2까지 포함한 공정한 두 아키텍처 비교



중요: Stage 2 weight를 공유하는 것이 아니라, 동일 recipe로 각각 별도 학습한다.

그림 5. 동일 Stage 2 recipe 로 각 아키텍처를 별도 학습하는 비교

11.1 같은 Stage 2 의 정확한 의미

"같은 Stage 2"는 하나의 policy weight 를 두 Stage 1 에 번갈아가 쓴다는 뜻이 아니다. Baseline foresight 와 Graph foresight 는 분포가 다를 수 있으므로, 각 Stage 1 을 고정한 뒤 동일 architecture 와 동일 training recipe 를 사용하여 각자의 Stage 2 를 별도로 학습해야 한다.

해야하는 것	하면 안되는 것
동일 seed 로 초기화한 두 policy 를 각각 학습	Baseline foresight 로 학습한 policy 에 graph foresight 를 평가 때만 교체
동일 batch 순서와 update 수 사용	한 모델만 더 오래 학습하거나 더 좋은 checkpoint 를 임의 선택
동일 rollout initial states 사용	각 모델에 다른 initial condition 사용

해야하는 것	하면 안되는 것
모든 hyperparameter 를 config diff 로 검증	"대충 비슷한 설정"으로 비교

11.2 반드시 동일하게 통제할 항목

범주	고정 내용
데이터	동일 demonstration, episode-level split, preprocessing
Stage 1 budget	동일 step, batch, optimizer, scheduler, common module initialization
Stage 2 architecture	동일 layer 수, hidden size, action horizon, conditioning 위치
Diffusion	동일 noise schedule, number of steps, sampler
Optimization	동일 optimizer, LR, weight decay, gradient clipping
Randomness	동일 seed set; 가능하면 paired seed 비교
Checkpoint	동일 validation metric 과 선택 rule
Evaluation	동일 task, rollout 수, initial states, maximum steps
Compute reporting	parameter 수, 학습 시간, inference latency, memory

11.3 주 비교와 보조 비교

구분	비교	해석
Primary	E0 Reimplemented CLaD vs E3 Correct-edge Graph-CLaD	Graph augmentation 의 전체 효과
Control	E1 Parameter-matched MLP adapter	추가 parameter 만으로 생긴 효과인지
Ablation	E2 No-edge GNN, E4 Shuffled-edge GNN	Relation edge 의 의미 기여
Diagnostic	Policy cross-evaluation	Representation mismatch 확인용, 주 결과로 사용하지 않음

12. 전체 실험 matrix 와 metric

12.1 실험 ID

ID	Stage 1 variant	Stage 2	핵심 목적
E0	Reimplemented CLaD	동일 recipe 로 별도 학습	주 baseline

ID	Stage 1 variant	Stage 2	핵심 목적
E1	CLaD + parameter-matched MLP adapter	동일 recipe	parameter 증가 통제
E2	CLaD + no-edge GNN	동일 recipe	message passing/graph container 효과
E3	CLaD + correct-edge GNN	동일 recipe	주 Graph-CLaD
E4	CLaD + shuffled-edge GNN	동일 recipe	올바른 relation 의미의 기여
E5 선택	Graph-only 또는 semantic 제거	동일 recipe	각 modality 정보량 진단

12.2 Stage 1 metric

Metric	보고 방식	좋은 결과의 의미
Latent prediction loss	validation 평균 및 seed 표준편차	미래 latent target 을 더 잘 맞춘다
Reconstruction loss	proprio/visual 별도	foresight 가 관측과 연결된 상태 유지
Object displacement MAE/MSE	전체 + relation-change subset	object movement 예측 개선
Relative position MSE	object-target pair 별	공간 관계 보존
Relation-change Macro F1	class 별 precision/recall 포함	희귀 relation 변화도 평가

12.3 Stage 2 metric

Metric	필수 보고
Average success rate	전체 task 평균과 seed 표준편차
Task-level success rate	각 LIBERO task 별 값
Relation-critical subset	유사 object, placement, containment 등 graph 기설과 관련된 task/구간
Training stability	policy loss curve, NaN 여부, best step
Efficiency	parameter 수, inference Hz 또는 latency, peak GPU memory

12.4 권장 통계

가능하면 3 개 이상의 seed 를 사용하고, 같은 seed 끼리 baseline 과 graph 모델을 paired comparison 한다. Success rate 는 rollout 수가 적으면 분산이 크므로 task 별 rollout 수와 confidence interval 을 함께 기록한다. 초기 feasibility 에서는 seed 수가 부족하더라도 단일 값만 강조하지 말고 정확한 rollout 수와 제한을 명시한다.

13. 데이터 split, leakage, seed 와재현성

13.1 Split 원칙

- Episode 단위로 train/validation/test 를 나눈다
- 같은 episode 에서 뽑은 모든 temporal window 는 하나의 split 에만 속해야 한다.
- Normalization 통계는 train split 에서만 계산한다
- Hyperparameter 선택은 validation 으로 하고 test 는 마지막에만 사용한다.
- Stage 1 과 Stage 2 가 같은 split 정의를 공유하도록 split manifest 파일을 고정한다.

13.2 Seed manifest

```
seeds:
  data_split: 100
  model_init: [0, 1, 2]
  batch_order: [0, 1, 2]
  rollout_initial_states: fixed_manifest.json
```

13.3 실험기록 필수 항목

항목	예시
Git commit	a1b2c3d
Config hash	stage2_e3_seed1.yaml + SHA
Data manifest	split_v1.json
Command	python train_stage2.py --config ...
Environment	torch/CUDA/Python/GPU
Checkpoint rule	best validation policy loss 또는 고정 step
Rollout manifest	task, seed, initial-state ID
Result file	metrics.csv, failure_cases.json

14. 단계별 Gate 와 일정

단계별 Gate: 다음 단계로 넘어갈 조건

Gate 0	코드 smoke test	shape/NaN/backward 확인 또는 정확한 blocker
Gate 1	Oracle graph	두 시점 시각화, ID 정렬, leakage test 통과
Gate 2	Offline graph probe	correct-edge가 MLP/no-edge/shuffled 대비 의미 있는 패턴
Gate 3	Graph-CLaD Stage 1	adapter OFF=baseline, 학습 안정성, latent target 개선
Gate 4	Stage 2 policy	baseline policy가 최소한 학습되고 rollout pipeline 검증
Gate 5	최종 비교	동일 recipe·여러 seed·task별 결과와 비용까지 보고

그림 6. 이전 단계의 검증 없이 다음 단계로 넘어가지 않는 Gate 구조

14.1 권장 단계와 종료 조건

단계	목표	종료 산출물
Phase 0	환경 코드 정리	제공 코드 import, synthetic forward/backward, shape 로그 또는 명확한 blocker
Phase 1	LIBERO state 조사	object ID/pose/gripper 접근 범위 문서화, JSON sample
Phase 2	Oracle graph dataset	두 시점 graph 시각화, unit test, episode split
Phase 3	Offline probe	MLP/no-edge/correct/shuffled 비교표
Phase 4	Graph-CLaD Stage 1	adapter OFF 동일성, Stage 1 학습 및 metric
Phase 5	Baseline Stage 2	재구성 CLaD 기반 policy 가 최소 rollout 학습
Phase 6	Graph Stage 2	동일 recipe 로 Graph-CLaD policy 학습
Phase 7	최종 평가	여러 seed, task 별 SR, efficiency, 실패 분석
Phase 8	발표·보고	주장 수준, 한계, 다음 단계 정리

14.2 현실적인 시간계획

Stage 2 전체 코드가 없으므로 기존 13 일 계획 안에 모든 단계를 안정적으로 끝내는 어렵다. 13 일은 Phase 0-4 의 feasibility 목표로 유지하고, Stage 2 를 추가한 전체 목표는 환경과 데이터 접근 상태에 따라 약 4-8 주 규모로 보는 것이 현실적이다. 아래 일정은 예시이며, 각 Gate 가 통과되지 않으면 날짜보다 원인 기록을 우선한다.

기간	권장 범위
1 주차	코드 분석, smoke test, LIBERO state 접근, graph spec
2 주차	Oracle dataset, MLP/GNN offline probe, ablation
3 주차	Graph-CLaD Stage 1 통합과 학습 안정화
4-5 주차	Stage 2 baseline 구현, diffusion training/rollout pipeline
5-6 주차	Graph Stage 2, 통제 비교
7-8 주차	여러 seed, 실패 분석, 문서와 발표 정리

15. Repository 와 구현 순서

15.1 권장 구조

```

project/
├── baseline_clad/
│   ├── Attentions.py
│   ├── MlpResNet.py
│   └── LatentDynamics.py
├── graph_clad/
│   ├── graph_spec.py
│   ├── oracle_extractor.py
│   ├── graph_dataset.py
│   ├── graph_encoder.py
│   └── graph_latent_dynamics.py
├── stage2/
│   ├── observation_encoder.py
│   ├── film_conditioner.py
│   ├── diffusion_policy.py
│   ├── scheduler.py
│   └── rollout.py
├── scripts/
│   ├── inspect_libero_state.py
│   ├── build_graph_dataset.py
│   ├── train_offline_probe.py
│   ├── train_stage1.py
│   ├── train_stage2.py
│   └── evaluate_rollout.py
├── configs/
├── tests/
├── docs/
└── results/

```

15.2 구현 순서

1. 원본 세 파일을 baseline_clad 에 그대로 보존하고 최초 commit 을 만든다
2. `test\_latent\_dynamics\_synthetic.py` 로 shape, loss scalar, backward, EMA 를 검사한다
3. LIBERO observation/state inspection script 를 작성한다
4. GraphSpec dataclass 와 extractor unit test 를 만든다
5. Episode-level offline dataset 과 split manifest 를 생성한다
6. MLP 와 GNN probe 를 동일 trainer 로 학습한다
7. GraphLatentDynamics 를 baseline subclass 또는 wrapper 형태로 구현한다
8. Stage 1 baseline/graph 학습 script 를 같은 config schema 로 통합한다
9. Stage 2 의 condition interface 를 먼저 고정한 뒤 diffusion network 를 구현한다
10. Baseline Stage 2 가 학습되는 것을 확인한 다음 Graph Stage 2 를 동일 recipe 로 실행한다

15.3 필수 unit test

Test	검사내용
test_baseline_smoke	forward/backward, NaN/Inf, output shape
test_ema	target parameter 가 online 을 momentum 으로 따라가는지
test_graph_identity	두 시점 동일 object ID 정렬
test_graph_invariance	node ordering 을 바꿔도 readout 이 일관적인지
test_no_future_leakage	future field 가 input tensor 에 포함되지 않는지
test_adapter_off	alpha=0 에서 baseline 과 Graph-CLaD output 동일
test_stage2_condition_shape	두 Stage 1 variant 가 동일 condition interface 제공
test_config_diff	비교 실험의 허용된 차이만 존재하는지

16. 결과 해석과 주장 수준

16.1 가능한 결과 조합

Offline probe	Stage 1 latent	Stage 2 SR	해석
개선	개선	개선	가장 강한 결과 relation 정보가 표현과 policy 까지 기여

Offline probe	Stage 1 latent	Stage 2 SR	해석
개선	개선	변화 없음	Representation 향상이 policy 가 이용할 수 있는 신호로 연결되지 않음
개선	변화 없음	변화 없음	독립 graph task 와 CLaD fusion 사이의 mismatch 가능
변화 없음	변화 없음	변화 없음	현재 graph 정의/target 에서 GNN 필요성 근거 부족
변화 없음	Stage 2 개선	개선	Offline metric 이 실제 control utility 를 측정하지 못했을 가능성 추가 검증 필요
악화	악화	악화	privileged graph 가 noise/shortcut 또는 fusion 방해로 작용

16.2 발표에서 강조할 것

- 전체 평균뿐 아니라 relation-change subset 과 유사 object task 결과를 함께 보여준다.
- Correct-edge 와 shuffled-edge 차이를 보여 graph relation 의 실제 기여를 설명한다.
- Stage 2 에서 성능이 오르지 않아도 Stage 1 까지의 결과와 blocker 를 분리하여 보고한다.
- 추가 parameter, 계산량, oracle privilege 를 숨기지 않는다.
- 공식 CLaD 수치는 참고로만 표시하고 직접 재현 수치처럼 표현하지 않는다.

17. 위험 요소와 대체 경로

위험	영향	대응
VLM pipeline 부재	Stage 1 real-data 입력을 만들 수 없음	한 frozen encoder 를 선택해 양쪽에 동일 적용, 논문 수치 재현 주장 금지
Stage 2 구현 난이도	Diffusion policy 가 학습되지 않음	작은 action diffusion smoke task → 단일 LIBERO task → 전체 task 순으로 확장
Oracle state 접근 제한	Demo 파일에는 pose 가 없고 live simulator 만 제공	Live simulator rollout 으로 state 기록하거나 접근 가능한 subset 부터 진행
Graph shortcut	Task ID 나 object name 만으로 target 을 외움	Episode/task split, feature ablation, shuffled relation 검사
Parameter 불공정	Graph model 만큼	Parameter-matched MLP 와 parameter/latency 보고
Rollout 분산	적은 rollout 로 결론 흔들림	고정 initial-state manifest, paired seeds, confidence interval
시간 초과	Stage 2 까지 완료 불가	Gate 별 산출물을 남기고 Stage 1 결과와 Stage 2 blocker 를 분리 보고

18. 지금 당장 해야 할 작업

18.1 오늘

- 새 repository 를 만들고 제공된 세 파일을 수정 없이 baseline_clad 에 보관한다
- 현재 코드의 synthetic smoke test 를 작성한다. 우선 $H=vl_dim=1024$, $V=2$ 라는 가정으로 실행하되 문서에 가정이라고 표시한다
- Loss 합산 wrapper 와 `model.update_ema()` 호출 위치를 임시 trainer 에 명시한다
- 중간 tensor hook 을 추가하되 baseline 동작을 바꾸지 않는다
- `docs/unknowns.md` 에 VLM, action dimension, Stage 2 세부 사항을 기록한다

18.2 다음 2-3 일

- LIBERO 환경 설치 및 observation key/object state 접근 확인
- GraphSpec 확장 node, edge, 좌표계, normalization, relation threshold
- 두 시점 graph JSON 과 시각화 생성
- Episode split manifest 작성
- MLP/GNN offline trainer skeleton 작성

18.3 Stage 2 를 시작해도 되는 시점

Stage 2 시작 조건

(1) Baseline Stage 1 과 Graph Stage 1 이 같은 interface 로 안정적으로 학습되고, (2) Graph adapter OFF 테스트가 통과하며, (3) offline probe 에서 graph 를 사용할 최소 근가가 있고, (4) action chunk dataset 과 rollout 환경이 준비되었을 때 Stage 2 를 시작한다

부록 A. Eq. (5)-(22) 수식 코드 실험 매핑

Eq.	논문 역할	현재 신규 구현	검증 포인트
5-6	State encoders	p_backbone, s_backbone	입력 feature 와 token shape 검증
7-8	Modality transition	p_dyn, s_dyn	과거 action 으로 현재 transition 표현
9	Asymmetric attention	latent_dyn(tp, ts)	query 방향 유지
10	Pooling	dyn_readout	z_dyn [B,H]
11-13	Foresight predictors	p_emb, s_emb	Stage 2 interface
14-16	EMA target	target backbone, update_ema	trainer 호출 구현

Eq.	논문 역할	현재 신규 구현	검증 포인트
17	Latent loss	loss_p, loss_s	normalization 차이 확인
18	Recon loss	loss_p_recon, loss_v_recon	관측 grounding
19	Combined objective	trainer 에서 구현	lambda_recon=0.1 은 논문 참고
20	Current obs encoding	Stage 2 새 구현	baseline/graph 동일
21	FiLM modulation	Stage 2 새 구현	foresight 와 현재 상태 결합
22	Diffusion policy loss	Stage 2 새 구현	동일 action diffusion recipe

부록B. 실험 기록 양식

```

experiment_id: E3_stage2_seed1
date: 2026-__-__
git_commit: _____
config: configs/e3_stage2_seed1.yaml
data_manifest: split_v1.json
stage1_checkpoint: _____
stage2_init_seed: 1
train_steps: _____
best_checkpoint_rule: _____

```

Stage 1 metrics:

```

latent_p: ____
latent_s: ____
object_displacement_mae: ____
relation_macro_f1: ____

```

Stage 2 metrics:

```

avg_success_rate: ____
task_success_rates: ____
rollout_count_per_task: ____
inference_latency: ____
peak_memory: ____

```

Failure notes:

```

- _____
- _____

```

부록C. 교수님께 보낼 진행 보고 초안

제목: [개별연구] Graph-CLaD Stage 2 까지의 통제 비교 연구 계획 공유

교수님 안녕하세요. 조은강입니다.

CLaD 연구자분께서 공유 가능한 Stage 1 핵심 코드 세 파일을 전달해 주셨고

현재 논문의 Eq. (5)-(19)와 실제 구현을 연결해 데이터 흐름과 누락 영역을 정리했습니다

공유받은 코드에는 전체 trainer, VLM 전처리, Stage 2 diffusion policy 가 포함되어 있지 않아

논문 수치 자체를 재현하기보다는 동일한 자체 재구현 환경에서 통제 비교를 진행하려고 합니다

먼저 LIBERO simulator 의 정확한 object identity 와 pose 를 이용해 oracle object graph 를 만들고 동일한 데이터 split 에서 parameter-matched MLP, no-edge GNN, correct-edge GNN, shuffled-edge GNN 을 비교하여 relation 정보가 미래 object displacement 와 relation change 예측에 실제로 필요한지 확인하겠습니다.

이 결과가 의미 있을 경우 graph representation 을 CLaD 의 semantic transition 에 residual adapter 로 결합하여 Stage 1 future latent prediction 을 비교하겠습니다. 최종적으로는 Baseline CLaD Stage 1 과 Graph-CLaD Stage 1 을 각각 고정한 뒤 architecture, initialization, optimizer, diffusion schedule, training step, 데이터와 평가 초기 상태를 동일하게 맞추어 Stage 2 policy 를 별도로 학습해 최종 LIBERO success rate 를 비교하는 것을 목표로 합니다.

이 비교는 논문에 보고된 공식 94.7%와의 직접 비교가 아니라, 동일한 재구성 조건에서 graph augmentation 의 효과를 검증하는 형태로 주장 범위를 제한하겠습니다. 단계별 결과와 blocker 를 정리해 다시 공유드리겠습니다.

감사합니다
조은강 드림

근거자료

1. CLaD: Planning with Grounded Foresight via Cross-Modal Latent Dynamics, 제공된 논문 PDF. 주요 근거 Sec. 4.1-4.3, Eq. (5)-(22), Sec. 5.1, 5.3, Limitations.
2. 제공 코드 `LatentDynamics.py`: Stage 1 class, EMA target, transition blocks, prediction/reconstruction losses.
3. 제공 코드 `Attentions.py`: CrossAttnLayer/Block, SelfAttnLayer/Block.
4. 제공 코드 `MlpResNet.py`: MlpResNet, FiLM_layer, FilmMLP.
5. 기존 `CLaD Object-Relation 연구계획` PDF: oracle graph feasibility, offline MLP-GNN 비교, residual fusion 제안
6. 본 문서의 Stage 2 구현 protocol, 공정 비교 matrix, Gate 및 repository 구조는 위 자료의 빈칸을 명시적으로 구분하여 작성한 [연구설계]이다.

한페이지 실행 요약 및 최종 체크리스트

최종 목표

Offline graph 효과를 확인한 뒤 Graph-CLaD Stage 1 을 구현하고, Baseline/Graph Stage 1 을 각각 고정하여 동일 Stage 2 architecture 와 recipe 로 policy 를 별도 학습한 뒤 LIBERO success rate 를 비교한다.

단계	핵심 산출물
1. Baseline 이해	수식 코드 매핑, synthetic smoke test, tensor shape
2. Oracle graph	object ID/pose 기반 graph, two-timestep visualization, episode split
3. Offline probe	MLP/no-edge/correct/shuffled edge 결과
4. Graph-CLaD Stage 1	adapter OFF=baseline, latent/offline metric
5. Baseline Stage 2	재구현 CLaD 용 diffusion policy 와 rollout
6. Graph Stage 2	동일 recipe 의 Graph-CLaD policy
7. 최종 비교	task 별 SR, 여러 seed, efficiency, failure analysis

최종 체크리스트

- ☐ Baseline 과 Graph-CLaD 의 차이가 graph branch 와 fusion 뿐인가?
- ☐ Stage 1 output interface 가 동일한가?
- ☐ 각 Stage 1 에 맞는 Stage 2 를 동일 recipe 로 별도 학습했는가?
- ☐ Data split, seed, batch order, checkpoint rule, rollout initial state 가 동일한가?
- ☐ Correct/no-edge/shuffled/parameter-matched control 이 있는가?
- ☐ Oracle privilege 와 공식 논문 수치 비재현을 명시했는가?
- ☐ Stage 1 metric 과 Stage 2 policy metric 을 모두 보고했는가?
- ☐ 성공뿐 아니라 실패 비용, blocker 를 기록했는가?

주장 문장

"동일한 재구현 조건에서 graph augmentation 의 효과를 평가했다" 라고 말한다. "공식 CLaD 94.7%를 능가했다"고 말하지 않는다.